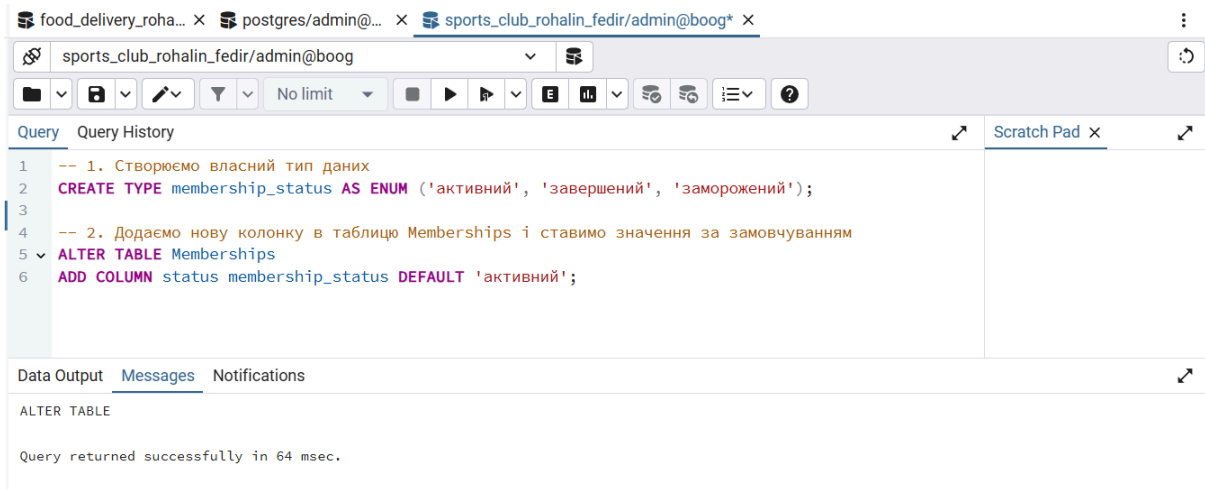


Звіт до лабораторної роботи №4

Виконав студент групи МІТ-31

Рогалін Федір

Додамо до таблиці абонементів (Memberships) статус. Абонемент може бути тільки "активний", "завершений" або "заморожений". (Рис. 1)



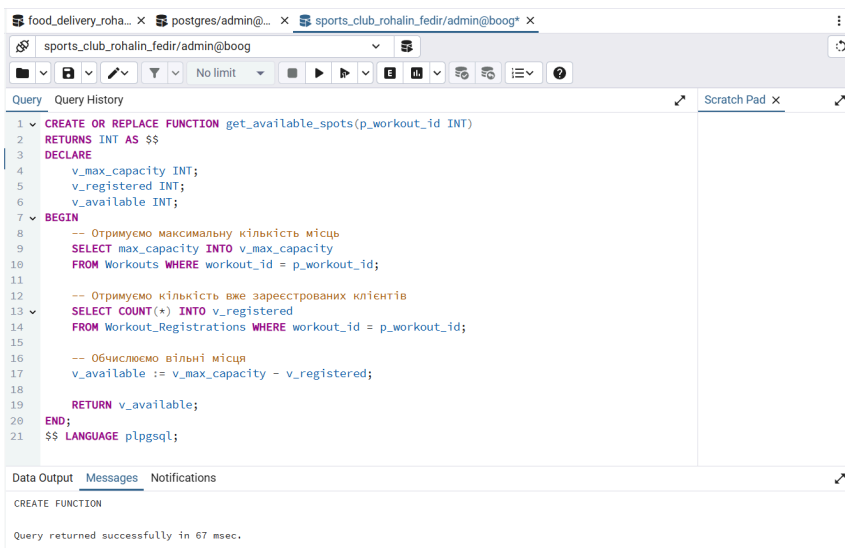
```
1 -- 1. Створюємо власний тип даних
2 CREATE TYPE membership_status AS ENUM ('активний', 'завершений', 'заморожений');
3
4 -- 2. Додаємо нову колонку в таблицю Memberships і ставимо значення за замовчуванням
5 ALTER TABLE Memberships
6 ADD COLUMN status membership_status DEFAULT 'активний';
```

ALTER TABLE

Query returned successfully in 64 msec.

Рис. 1.

Створимо функцію для адміністратора клубу, яка буде рахувати кількість вільних місць на конкретне тренування (віднімаючи від максимальної місткості кількість вже зареєстрованих людей). (Рис. 2)



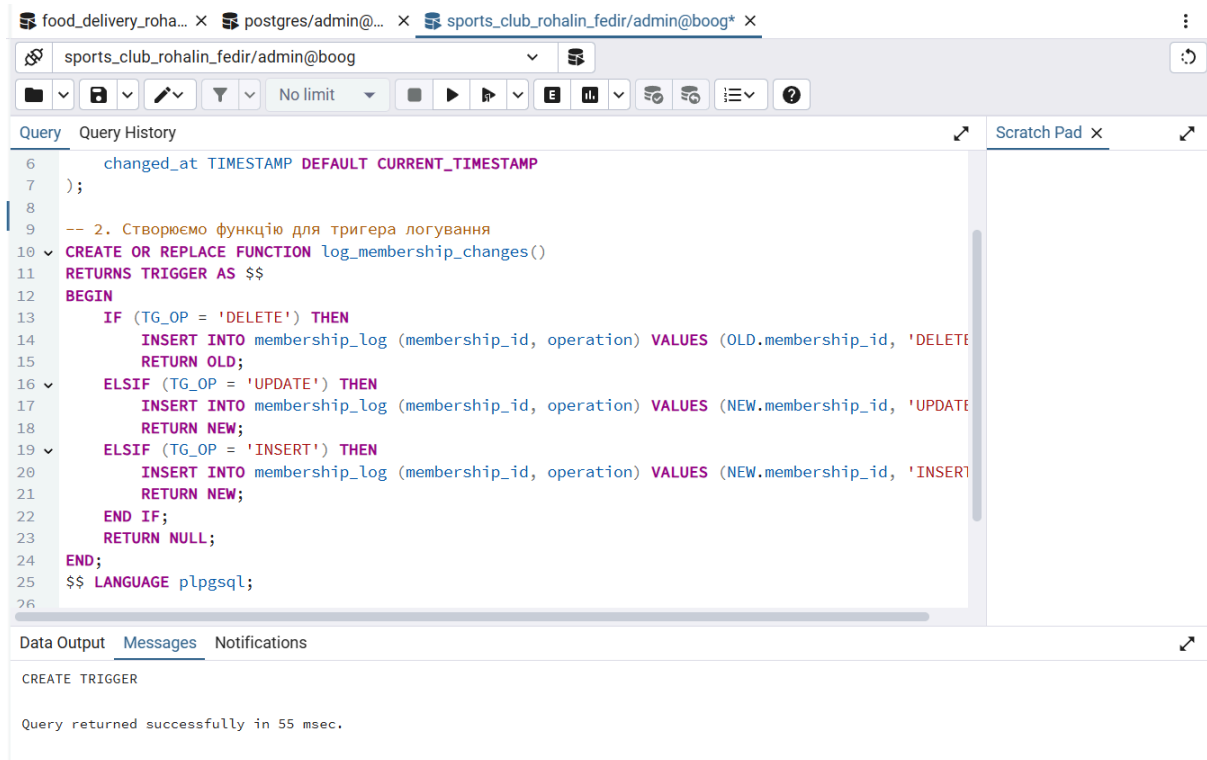
```
1 CREATE OR REPLACE FUNCTION get_available_spots(p_workout_id INT)
2 RETURNS INT AS $$
3 DECLARE
4     v_max_capacity INT;
5     v_registered INT;
6     v_available INT;
7 BEGIN
8     -- Отримуємо максимальну кількість місць
9     SELECT max_capacity INTO v_max_capacity
10    FROM Workouts WHERE workout_id = p_workout_id;
11
12     -- Отримуємо кількість вже зареєстрованих клієнтів
13     SELECT COUNT(*) INTO v_registered
14    FROM WorkoutRegistrations WHERE workout_id = p_workout_id;
15
16     -- Обчислюємо вільні місця
17     v_available := v_max_capacity - v_registered;
18
19     RETURN v_available;
20 END;
21 $$ LANGUAGE plpgsql;
```

CREATE FUNCTION

Query returned successfully in 67 msec.

Рис. 2.

Тригер 1: Аудит (Логування) Створимо таблицю-журнал, куди PostgreSQL буде автоматично записувати, хто і коли змінював дані про абонементи. (Рис 3.)



```
6      changed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
7  );
8
9  -- 2. Створюємо функцію для тригера логування
10 CREATE OR REPLACE FUNCTION log_membership_changes()
11 RETURNS TRIGGER AS $$
12 BEGIN
13     IF (TG_OP = 'DELETE') THEN
14         INSERT INTO membership_log (membership_id, operation) VALUES (OLD.membership_id, 'DELETE');
15         RETURN OLD;
16     ELSIF (TG_OP = 'UPDATE') THEN
17         INSERT INTO membership_log (membership_id, operation) VALUES (NEW.membership_id, 'UPDATE');
18         RETURN NEW;
19     ELSIF (TG_OP = 'INSERT') THEN
20         INSERT INTO membership_log (membership_id, operation) VALUES (NEW.membership_id, 'INSERT');
21         RETURN NEW;
22     END IF;
23     RETURN NULL;
24 END;
25 $$ LANGUAGE plpgsql;
```

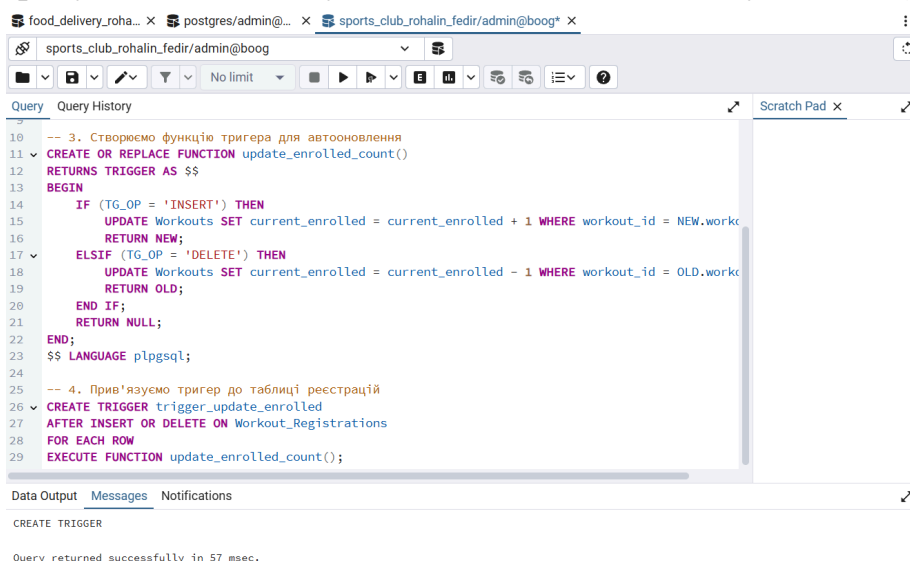
Data Output Messages Notifications

CREATE TRIGGER

Query returned successfully in 55 msec.

Рис. 3.

Тригер 2: Автоматичне оновлення пов'язаних таблиць Давай додамо в таблицю Workouts поле current_enrolled (поточна кількість записаних). Воно буде автоматично збільшуватись, коли хтось записується на тренування, і зменшуватись, коли хтось скасовує запис. (Рис. 4)



```
10 -- 3. Створюємо функцію тригера для автооновлення
11 CREATE OR REPLACE FUNCTION update_enrolled_count()
12 RETURNS TRIGGER AS $$
13 BEGIN
14     IF (TG_OP = 'INSERT') THEN
15         UPDATE Workouts SET current_enrolled = current_enrolled + 1 WHERE workout_id = NEW.workout_id;
16         RETURN NEW;
17     ELSIF (TG_OP = 'DELETE') THEN
18         UPDATE Workouts SET current_enrolled = current_enrolled - 1 WHERE workout_id = OLD.workout_id;
19         RETURN OLD;
20     END IF;
21     RETURN NULL;
22 END;
23 $$ LANGUAGE plpgsql;
24
25 -- 4. Прив'язуємо тригер до таблиці реєстрацій
26 CREATE TRIGGER trigger_update_enrolled
27 AFTER INSERT OR DELETE ON Workout_Registrations
28 FOR EACH ROW
29 EXECUTE FUNCTION update_enrolled_count();
```

Data Output Messages Notifications

CREATE TRIGGER

Query returned successfully in 57 msec.

Рис. 4.

Data Output (Рис. 5.)

The screenshot shows a PostgreSQL query editor interface. The top bar indicates the user is logged in as 'admin@boog' on the 'sports_club_rohalin_fedir' database. The query editor contains a SQL script with 16 lines, including comments in Ukrainian and SQL commands. The 'Data Output' tab is active, showing a table with 5 columns: log_id, membership_id, operation, and changed_at. The table contains one row of data.

Query:

```

1  -- ТЕСТ 1: Перевірка роботи функції
2  -- Перевіряємо вільні місця на тренування з id = 1
3  SELECT title, max_capacity, get_available_spots(1) AS available_spots
4  FROM Workouts WHERE workout_id = 1;
5
6  -- ТЕСТ 2: Перевірка тригера автооновлення
7  -- Додаємо новий запис на тренування 1 для клієнта 3
8  INSERT INTO Workout_Registrations (workout_id, member_id) VALUES (1, 3);
9  -- Дивимось, чи збільшилась колонка current_enrolled у таблиці Workouts
10 SELECT title, max_capacity, current_enrolled FROM Workouts WHERE workout_id = 1;
11
12 -- ТЕСТ 3: Перевірка тригера логування та нового типу ENUM
13 -- Змінюємо статус абонементу на "заморожений"
14 UPDATE Memberships SET status = 'заморожений' WHERE membership_id = 1;
15 -- Перевіряємо, чи з'явився запис про це у логах
16 SELECT * FROM membership_log;

```

Data Output:

	log_id [PK] integer	membership_id integer	operation character varying (10)	changed_at timestamp without time zone
1	1	1	UPDATE	2026-05-05 22:54:04.513787

Рис. 5.

Висновки:

Під час виконання лабораторної роботи було створено користувацький тип ENUM для контролю статусів абонементів. Реалізована функція `get_available_spots()` дозволяє інкапсулювати бізнес-логіку підрахунку вільних місць на рівні БД, що зменшує навантаження на бекенд. Тригер логування забезпечує надійний аудит фінансових змін, а тригер автооновлення підтримує актуальну статистику записів, пришвидшуючи виконання запитів на вибірку розкладу.